indicating they do not contribute meaningfully toward the correct final answer. When the model identifies a more promising path and self-corrects, subsequent steps will show positive information gain, signaling productive progress.

This adaptability to varied reasoning structures is empirically demonstrated in our experiments (Section 5), where we analyse reasoning traces from the MATH/PRM dataset. Steps that human annotators labelled as uninformative or irrelevant consistently show low information gain under our metric, while correct and meaningful steps exhibit high information gain. Thus, our method provides reliable step-wise evaluation regardless of whether the reasoning follows a linear chain or involves branched/exploratory patterns.

We further formalize this remark using our framework in Appendix B.4.

4 Related Works

Evaluation of CoT reasoning Several recent works propose methodologies for evaluating CoT reasoning [11, 15, 16, 21, 28, 32, 35, 37]. For example, [16] verifies individual steps in a model's CoT reasoning by generating multiple LLM responses per prompt and comparing correct responses with incorrect ones.

Similarly, [29, 30] use a fine-tuned LLM to decode multiple reasoning paths from each step and check the correctness of these reasoning paths. However, as we show in our experiments, approaches which simply rely on the correctness of the final output are not sound in general and can lead to false positives. Moreover, these solutions may not be plausible for problems of high difficulty where correct LLM responses might be scarce.

Formalising CoT framework The formalisation of LLM reasoning remains an active area of research. Most notably [10] introduces a formal framework for LLMs and is a key source of inspiration behind our formalism. Additionally, [9] theoretically examines the expressivity of LLMs with CoT in solving mathematical and decision-making problems, focusing on the transformer architecture's implications on accuracy. Besides this, [36] provides a formal definition of hallucinations, but does not consider CoT reasoning specifically.

Reward modelling Outcome-based reward models (ORM) [6, 11, 18] predict the probability of reaching the correct final answer based on a model's intermediate CoT steps. While they avoid requiring correct intermediate demonstrations, we show in Section 5 that they are unsound for detecting CoT reasoning errors. Step-wise ORM (SORM) [11] extends ORM by estimating the probability of an 'optimal' model reaching a correct answer but requires training a larger, more capable model than the base model.

Process-based reward modelling (PRMs) [18, 27] is an alternative approach which directly predicts the correctness of intermediate CoT reasoning steps. Likewise, various other approaches rely on annotated CoT datasets for benchmarking [1, 14, 19–21, 34, 35, 37]. While these benchmarks and methodologies aid LLM reasoning, collecting annotated data is costly and not easily scalable. In contrast, our approach evaluates an LLM's CoT reasoning without human-annotated CoT data.

5 Experiments

In this section, we demonstrate our framework's utility, dubbed Information-Gain (IG) and compare against two baselines for detecting errors in a model's CoT reasoning. Here we assume access only to the model's CoT generations $X_0, X_1^M, \ldots, X_T^M$ and correct final answers Y.

Outcome Reward Model (ORM) [11] This involves training a classifier, denoted as f_{ORM}, which takes as input model generations up to any step t in its CoT reasoning, X_t^M, and predicts the probability of the model's final answer being correct, i.e.

$$f_{\text{ORM}}(X_t^M) \approx \mathbb{P}(Y^M = Y \mid X_t^M). \tag{6}$$

Here, if we observe that this probability of correctness drops significantly after step t, i.e. $f_{\text{ORM}}(X_t^M) \gg f_{\text{ORM}}(X_{t+1}^M)$, this indicates that the model applies task λ_{t+1} incorrectly.

Math-Shepherd (MS) [29] This method quantifies the potential for a given reasoning process X_t^M by using a 'completer' model to generate N completions of each reasoning process starting from step t, $\{(X_t^M, X_{t+1,j}^M, \ldots, X_{T,j}^M, Y_j^M)\}_{j \leq N}$, where Y_j^M denotes the final answer reached in the j'th completion. Then, we estimate the potential of this step based on the proportion of correct answers among the N completions:

$$f_{\text{MS}}(X_t^M) := \sum_{j=1}^{N} \mathbb{1}(Y_j^M = Y)/N. \tag{7}$$

For a fair comparison, we do not assume access to a 'verifier' model more capable than our base model. Therefore, we use the base model as the completer model in our experiments.

5.1 Toy data experiments

We first consider a toy setting where we control model behaviour across tasks. Prompts consist of an integer vector $Z_0 \in \mathbb{Z}^5$ sampled from a given distribution. The task λ comprises five steps, $\lambda = \lambda_1 \circ \ldots \circ \lambda_5$, where each sub-task λ_i transforms $Z_{i-1} \in \mathbb{Z}^5$ into $Z_i \in \mathbb{Z}^5$. The correct final answer Y is Z_5. Additional details on data generation and sub-tasks are in Appendix C.1.

Generating the dataset To investigate partial unidentifiability for a given task λ_i we modify the obtained dataset by introducing 'noise' at step i. In other words, the task λ_i is applied incorrectly on a subset of the data, whereas all other tasks are always applied correctly. This represents a model which sometimes fails at step i and we use 'LLM_i' to denote this model in this experiment. We repeat this procedure for all tasks λ_i for $i \in \{1, \ldots, 5\}$ which yields 5 LLMs $\{\text{LLM}_1, \ldots, \text{LLM}_5\}$.

To assess robustness, we introduce a special case in LLM_3, where task λ_3 is applied incorrectly iff the output after λ_2 falls in a set $\mathcal{S}$. This deliberate choice highlights a pitfall of existing baselines and contrasts with other LLMs, where errors occur randomly. In other words, λ_3's correctness depends on λ_2's output. For details, see Appendix C.1.2.

5.1.1 Results

Figure 3 shows how the different baselines quantify the correctness of the different tasks for the 5 different LLMs under consideration. This figure only considers samples where the final answer of the LLM was incorrect, i.e. $Y^M \neq Y$. For our method (IG), Figure 3a shows the information-gain across the different steps for each LLM. Likewise, Figure 3b presents the results for ORM and shows how the average probability of correctness in (6) changes across the different steps, whereas, for Math-Shepherd, Figure 3c shows the proportion of correct completions starting after each step (7). Here, any significant drop in the plotted values indicate an incorrect application of a task.

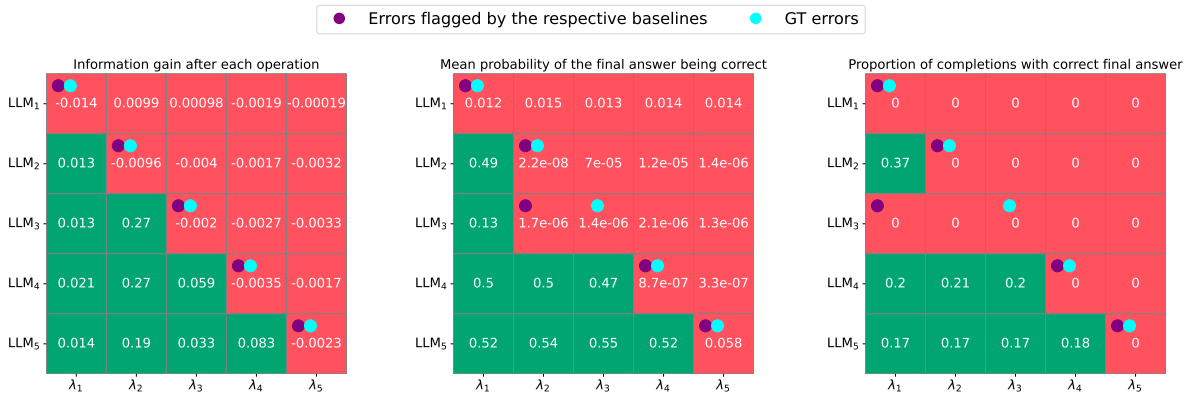
Information-gain rightly quantifies step-wise correctness We observe that the information-gain remains positive for each LLM until we encounter an incorrect reasoning step, at which point it drops to negative values. Therefore, our method can identify the incorrectly executed task for each LLM under consideration. We used a GPT-2 supervisor model to estimate information-gain.

Pitfall of the baselines While ORM and Math-Shepherd usually identify incorrect reasoning steps, they fail for LLM_3. This is because λ_3 is misapplied iff the output after λ_2 lies in $\mathcal{S}$. Thus, the classifier can predict the final output's correctness at λ_2 by checking if Z_2 lies in $\mathcal{S}$, leading to overconfidence in error detection at λ_2 instead of λ_3.

Similarly, with Math-Shepherd for LLM_3 (using the same model as a completer), a completion is incorrect if the output after λ_2 lies in $\mathcal{S}$. Here, all completions fail, regardless of the starting step, making it impossible to pinpoint where LLM_3 goes wrong.

Table 1 Sample-wise classification of a sub-task for LLM_3.

	Acc ↑	TPR ↑	FPR ↓
IG (Ours)	**0.96**	0.98	**0.06**
ORM	0.77	0.98	0.54
MS	0.60	**1.0**	1.0



(a) Our results **(b)** ORM results **(c)** Math-Shepherd results

Figure 3 Heatmaps (left) quantifying the correctness of different sub-tasks for the 5 LLMs using the different baselines, and the associated classification metrics (right). Red color in the heatmaps indicates a significant drop in the plotted metrics (an incorrectly executed sub-task).

Sample-wise detection We also use the different baselines for sample-wise detection of erroneous steps, as outlined in Section 3.3. A step is classified as incorrect if a baseline's metric falls below a threshold. Table 1 presents the results for LLM_3, with optimal thresholds chosen from a held-out dataset. Our method achieves significantly higher accuracy and fewer false positives than the baselines, making it more reliable for sample-wise error detection.

5.2 Arithmetic operations on Llama-3-8B

Following our toy experiments, we now evaluate our framework in a more realistic setting using the Llama-3-8B model [7]. We focus on a simple arithmetic task that involves both multiplication and addition. The goal is to assess the model's performance on each operation.

Experimental setup We sample two integers x and y uniformly from $[1, 10^5)$. The prompt to the model is:

Prompt: "*x = {x}, y = {y}, Please calculate the following: 1. $3x$, 2. $2y$, 3. $3x + 2y$* "

Model Accuracy The model's accuracy across steps is:

$$\text{Step 1: } 80\%, \qquad \text{Step 2: } 98\%, \qquad \text{Step 3: } 42\%.$$

Most failures occur in Step 3, which involves adding previously computed values. Analyzing the (x, y) distribution where the model is incorrect (Figure 4), we find that errors mainly arise when one variable is large and the other is small. This suggests that correctness depends heavily on (x, y), making it difficult for baselines to pinpoint the erroneous step in the model's CoT reasoning.

5.2.1 Results

Our method We trained the supervisor model by fine-tuning a Llama-3-8b model using Low Rank Adaptation (LoRA) [13]. Table 2 shows that there is a significant drop in information-gain at step 3 relative to steps 1 and 2, demonstrating that our method correctly identifies that the failure mainly occurs at step 3.

Outcome Reward Model (ORM) For ORM, the mean probability of correctness (Table 2) remains unchanged at each step. Figure 4 suggests this is because ORM predicts correctness based solely on x and y in the prompt. Crucially, its confidence remains constant even as intermediate reasoning steps are added, preventing it from distinguishing the model's performance at different steps.

Math-Shepherd (MS) Table 2 shows the proportion of correct completions for MS. While this is low at step 3, only 5-7% of completions from steps 1 and 2 yield a correct output, despite errors mostly occurring at step